

STAR Project

Comprehensive Gap Analysis & Implementation Roadmap

Branch: feat/gap-analysis-and-planning

Date: February 18, 2026

Status: Research Complete – Ready for Implementation

Simultaneous Tracking and Robotics (STAR) is a distributed robotics platform with custom PCB hardware, Renesas RX72N motor control firmware, Raspberry Pi 5 control system, and Protocol Buffers communication.

Controller:	Raspberry Pi 5 (ROS2 Jazzy)
Motor MCU:	Renesas RX72N (4MB Flash, 512KB SRAM, ThreadX)
Motors:	4x 6V brushed DC gearmotors, 210 RPM, 341 PPR
Drivers:	DRV8243S H-bridge with current sensing
Sensors:	RPLiDAR C1, 4x HC-SR04, DS18B20, BQ4050 BMS
Protocol:	nanopb + CRC-32 over 10 Mbps SPI / USB CDC
UI:	TypeScript/React via WebSocket gateway

Contents

1	Executive Summary	3
1.1	Completion Matrix	3
1.2	Critical Blockers (Must Fix Before Robot Moves)	4
2	Firmware Gaps (RX72N)	5
2.1	Gap 1: WireMessage Dispatcher (CRITICAL)	5
2.2	Gap 2: OTA Firmware Update Handler (HIGH)	6
2.3	Gap 3: NVS Configuration Persistence (HIGH)	6
2.4	Gap 4: BMS Telemetry Integration (MEDIUM)	7
2.5	Gap 5: RPLiDAR C1 Integration (LOW – deferred)	7
2.6	Firmware Status Summary	7
3	Gateway Gaps (star-gateway, Go)	8
3.1	Gap 1: FirmwareUpdateService (CRITICAL, 0% Complete) . .	8
3.2	Gap 2: E-Stop Priority Framing (HIGH)	9
3.3	Gap 3: SetPIDGains Silently Dropped (HIGH)	9
3.4	Gateway Service Status Summary	9
4	ROS2 Gaps (star-ros2)	9
4.1	Gap 1: star_spi_bridge Hardware Testing (CRITICAL)	10
4.2	Gap 2: Zero-Velocity Safety Shutdown (HIGH)	10
4.3	Gap 3: star_bringup Launch Files (HIGH, 5% Complete)	10
4.4	ROS2 Package Status Summary	11
5	UI Gaps (star-ui, TypeScript/React)	11
5.1	The Core Problem: Browser Cannot Access gRPC	12
5.2	Gap 1: Emergency Stop Button (CRITICAL – Safety)	12
5.3	Gap 2: Telemetry Display (CRITICAL)	12
5.4	UI Component Status	12
6	Protocol Buffer Gaps (star-proto)	12
6.1	Gap 1: motor_status max_count Mismatch (HIGH – Data Trun- cation Bug)	13
6.2	Gap 2: FirmwareChunk Missing in wire.proto (HIGH)	13
7	CI/CD Gaps	14
7.1	New Workflow: .github/workflows/firmware.yml	14
7.2	New Workflow: .github/workflows/ui.yml	14
8	Open Issue Branches Status	15
9	Critical Path to First Robot Motion	15
10	Hardware Gaps	17
10.1	Hardware Pinout Verification Checklist	17
11	Testing Gaps	18
11.1	Testing Priority Matrix	18

11.2 Local Test Runner (All Components)	18
12 Virtual RX72N Simulator Enhancements	19
12.1 Motor Dynamics Model	19
12.2 Fault Injection API	20
13 Implementation Roadmap	20
13.1 Immediate Actions (This Week)	21
13.2 Short-Term (Next 2-4 Weeks)	21
13.3 Medium-Term (1-2 Months)	21
13.4 Long-Term (2-6 Months)	21
14 Appendix: System Architecture	22
14.1 Communication Flow	22
14.2 Firmware Task Priority Table	22
14.3 Motor PID Parameters (from MATLAB)	22
14.4 SPI Frame Protocol	22

Executive Summary

The STAR project has a strong foundation with excellent firmware, communication infrastructure, and Protocol Buffers definitions. However, significant gaps exist in the **integration layer** – the “glue code” connecting all subsystems for real robot operation.

Overall System Completion: $\approx 65\%$

Completion Matrix

Component	%	Status	Notes
RX72N Firmware Core	100	Complete	All 9 tasks, 87 unit tests
Firmware WireMessage Dispatch	0	Missing	Blocks all commands
Firmware OTA Handler	0	Missing	Blocks firmware updates
Firmware NVS Config	0	Missing	PID not persistent
Gateway Transport/HARQ/FEC	100	Complete	Production ready
Gateway MotorControlService	80	Partial	E-Stop priority, SetMotorPower
Gateway TelemetryService	90	Partial	GetSystemStatus stubbed
Gateway BatteryService	100	Complete	80% test coverage
Gateway ConfigurationService	85	Partial	SetRetransmitConfig missing
Gateway FirmwareUpdateService	0	Missing	All 10 RPCs are stubs
Gateway GatewayService	85	Partial	SetPIDGains silently dropped
Proto Schemas	100	Complete	All messages defined
nanopb Options	85	Partial	motor_status max_count:2 bug
ROS2 star_gateway_bridge	100	Complete	Telemetry forwarding works
ROS2 star_spi_bridge	85	Partial	Code complete, hardware untested
ROS2 star_safety_monitor	40	Partial	3 of 10 tests skipped
ROS2 star_bringup	5	Missing	Empty skeleton only

(continued from previous page)

Component	%	Status	Notes
ROS2 SLAM Stack	0	Missing	RTAB-Map, Nav2 not configured
UI Gamepad Control	100	Complete	Working
UI Telemetry Display	0	Missing	No gRPC-Web bridge
UI Battery Status	0	Missing	-
UI Emergency Stop Button	0	Missing	Safety critical
UI Configuration Panel	0	Missing	-
CI/CD Firmware	0	Missing	No firmware CI workflow
CI/CD UI Build	0	Missing	No UI CI workflow
CI/CD Documentation	0	Missing	No LaTeX/Doxygen CI
Virtual RX72N Simulator	95	Usable	No motor dynamics
MATLAB PID Workflow	100	Complete	Kp=0.286, Ki=8.01, Kd=0
Hardware PCB Design	100	Complete	Gerbers ready for fab

Critical Blockers (Must Fix Before Robot Moves)

1. Firmware WireMessage Dispatcher (CRITICAL)

File: e2-studio-star-rx72n-firmware/src/tasks/comm_task.c

Impact: ALL commands from RPi5 (velocity, E-Stop, PID gains, config) are silently dropped after decoding.

Effort: 4-6 hours

2. UI Has No Telemetry Display (CRITICAL)

Root Cause: Browser cannot access Go gateway gRPC services directly. Needs WebSocket bridge.

Impact: Operator is completely blind to robot status. Cannot safely operate.

Effort: 10-30 hours (bridge + UI components)

3. ROS2 SPI Bridge Untested on Hardware (HIGH)

File: star-ros2/src/star_spi_bridge/

Impact: First physical robot motion requires this to work.

Effort: 4-8 hours on hardware

Firmware Gaps (RX72N)

The RX72N firmware has excellent core implementation – all 9 ThreadX tasks are complete, all hardware drivers work, and the HARQ/FEC/CRC-32 communication stack is production-ready. The missing piece is the **integration layer** between the communication task and application tasks.

Gap	Severity	Effort	Impact
WireMessage dispatcher	CRITICAL	4–6 hrs	ALL RPi5 commands blocked
OTA firmware update handler	HIGH	2–3 days	Firmware updates blocked
NVS configuration persistence	HIGH	4–6 hrs	PID settings lost on reboot
BMS telemetry integration	MEDIUM	4–6 hrs	Battery status invisible
RPLiDAR C1 integration	LOW	2–3 days	SLAM impossible
Host I2C (RIIC0) application	LOW	1–2 days	Future sensor expansion

Gap 1: WireMessage Dispatcher (CRITICAL)

The `comm_task.c` receives SPI/USB frames, decodes them via `nanopb`, but has **no dispatch logic** to route the decoded `star_v1_WireMessage` to the correct handler. Every command from the RPi5 is silently dropped.

```
static rx_err_t internal_dispatch_wire_message(
    const star_v1_WireMessage* msg)
{
    RX_CHECK_NULL_PTR(msg, k_rx_err_null_ptr);

    switch (msg->which_payload) {
    case star_v1_WireMessage_velocity_command_tag:
        return motor_control_handle_velocity_command(
            &msg->payload.velocity_command);

    case star_v1_WireMessage_emergency_stop_command_tag:
        return motor_control_handle_emergency_stop(
            &msg->payload.emergency_stop_command);

    case star_v1_WireMessage_pid_config_tag:
        return config_handle_pid_update(&msg->payload.pid_config);

    case star_v1_WireMessage_retransmit_config_tag:
        return config_handle_retransmit_update(
            &msg->payload.retransmit_config);

    default:
        rx_log_warn(k_tag_comm, "Unknown WireMessage type: %d",
            (int)msg->which_payload);
        return k_rx_ok; /* Forward compatible -- not fatal */
    }
}
```

```

    }
}

```

Listing 1: Required dispatcher implementation in comm_task.c

Handler signatures required in task headers:

```

rx_err_t motor_control_handle_velocity_command(
    const star_v1_VelocityCommand* cmd);
rx_err_t motor_control_handle_emergency_stop(
    const star_v1_EmergencyStopCommand* cmd);
rx_err_t motor_control_handle_power_command(
    const star_v1_MotorPowerCommand* cmd);

```

Listing 2: New public API in motor_control_task.h

Required Unity tests (tests/test_comm_task_dispatch.c):

- Velocity command routes to motor handler
- E-Stop command routes with highest priority
- Unknown type logs warning but does not crash
- NULL message returns k_rx_err_null_ptr
- PID config routes to config handler

Gap 2: OTA Firmware Update Handler (HIGH)

The Gateway has 10 RPC methods for OTA firmware update – all return codes.Unimplemented. The firmware has no OTA handler. The RX72N has dual-bank flash (documented in PR #305 linker fix), enabling live OTA.

Required state machine:

```

typedef enum : uint8_t {
    k_ota_state_idle           = 0, /**< No update in progress */
    k_ota_state_starting      = 1, /**< BeginUpdate received */
    k_ota_state_receiving     = 2, /**< WriteChunk frames arriving */
    k_ota_state_validating    = 3, /**< Checking CRC-32 + size */
    k_ota_state_ready         = 4, /**< Validated, ready to reboot */
    k_ota_state_error         = 5, /**< Error occurred */
} ota_state_t;

```

Listing 3: OTA state machine types

New library required: e2-studio-star-rx72n-firmware/libs/rx_ota/

Gap 3: NVS Configuration Persistence (HIGH)

PID gains and system configuration are received but not saved. On reboot all tuned parameters are lost. The RX72N has 64KB Data Flash (separate from code flash) ideal for NVS.

```

rx_err_t rx_nvs_write(const char* key, const void* data, uint16_t len);
rx_err_t rx_nvs_read(const char* key, void* data, uint16_t* len);
rx_err_t rx_nvs_delete(const char* key);
rx_err_t rx_nvs_erase_all(void); /* Factory reset */

```

Listing 4: Required NVS API (libs/rx_nvs/inc/rx_nvs.h)

Gap 4: BMS Telemetry Integration (MEDIUM)

The `bms_monitor_task` correctly detects battery faults and triggers E-Stop via IRQ13, but the BMS data (SOC, cell voltage, current) is **not included in the telemetry protobuf** sent to the RPi5. The operator cannot see battery state.

Required additions to `bms_monitor_task.h`:

```
uint32_t bms_get_voltage_mv(void);
uint32_t bms_get_current_ma(void);
uint8_t  bms_get_soc_percent(void);
int16_t  bms_get_temperature_deci_celsius(void);
bool     bms_is_charging(void);
uint32_t bms_get_fault_flags(void);
```

Listing 5: Thread-safe BMS getter API

Gap 5: RPLiDAR C1 Integration (LOW - deferred)

The RPLiDAR C1 is in the project hardware list but has zero firmware integration. Required: UART driver library (`libs/rx_rplidar/`), task (`lidar_scan_task.c`), pin assignment (SCI1 or SCI2), and LiDAR scan data in telemetry proto. **Estimated effort: 2-3 days.**

Firmware Status Summary

Module	Status
Motor Control (PID @ 100 Hz, 4 motors)	Complete
Communication Stack (SPI/USB, HARQ, FEC, CRC-32)	Complete
WireMessage Dispatch	NOT IMPLEMENTED
OTA Firmware Update	NOT IMPLEMENTED
NVS Configuration	NOT IMPLEMENTED
BMS Data Streaming (SOC/voltage)	Faults only
Temperature Sensing (DS18B20 @ 1 Hz)	Complete
Obstacle Detection (HC-SR04 IRQ @ 50 Hz)	Complete
Watchdog Monitor (IWDT @ 100 Hz)	Complete
LED Status Task (@ 20 Hz)	Complete
RPLiDAR C1 Driver	NOT IMPLEMENTED
Host I2C Sensors (RIIC0)	Reserved for future

Gateway Gaps (star-gateway, Go)

The gateway's transport and infrastructure layers are excellent and production-ready. The service layer has significant gaps – most critically FirmwareUpdateService which is 0% implemented.

Gap	Severity	Location	Effort
FirmwareUpdateService (10 RPCs)	CRITICAL	firmware.go	20-30 hrs
E-Stop priority framing	HIGH	motor_control.go:90	10 hrs
SetMotorPower not implemented	HIGH	motor_control.go:123	10 hrs
SetPIDGains not forwarding	HIGH	gateway_service.go:205	3-5 hrs
SetRetransmitConfig missing	MEDIUM	configuration.go	2 hrs
GetSystemStatus stub	LOW	telemetry.go:261	2 hrs

Gap 1: FirmwareUpdateService (CRITICAL, 0% Complete)

All 10 RPC methods in `internal/service/firmware.go` follow this pattern:

```
func (s *FirmwareService) BeginUpdate(ctx context.Context,
    req *starv1.BeginUpdateRequest) (*starv1.BeginUpdateResponse, error) {
    // TODO: Implement BeginUpdate
    return nil, status.Error(codes.Unimplemented, "not implemented")
}
```

Listing 6: Current state of `firmware.go` – all methods unimplemented

Affected RPCs: `BeginUpdate`, `WriteChunk`, `StreamChunks`, `FinalizeUpdate`, `AbortUpdate`, `GetUpdateProgress`, `StreamUpdateProgress`, `Reboot`, `Rollback`, `MarkValid`.

Required Go session management:

```
type otaSession struct {
    sessionID    string
    totalSize    uint64
    bytesWritten uint64
    expectedCRC  uint32
    startTime    time.Time
    mu           sync.Mutex
}
```

Listing 7: Session management struct for OTA

Also requires addition of `FirmwareChunk` to `wire.proto` `WireMessage` oneof (see Section 6).

Gap 2: E-Stop Priority Framing (HIGH)

Emergency stop commands use `PriorityNormal` in the HARQ layer. Under heavy telemetry load, an E-Stop could be delayed by queued normal frames – a safety hazard.

Fix: Add `PriorityEmergency` constant to the HARQ client and use it in `EmergencyStop()` method.

Gap 3: SetPIDGains Silently Dropped (HIGH)

`gateway_service.go:205` has a TODO: “Forward to ROS2 service when available.” Currently PID gain changes from the UI are accepted but never forwarded to the firmware. *Fix:* Send a `PidConfig WireMessage` directly via HARQ to the RX72N.

Gateway Service Status Summary

Service / Method	File	Status
SetVelocity	<code>motor_control.go</code>	Working
EmergencyStop	<code>motor_control.go</code>	Missing priority
SetMotorPower	<code>motor_control.go:139</code>	Implemented
StreamEncoders	<code>motor_control.go</code>	Working
GetTelemetry	<code>telemetry.go</code>	Working
StreamTelemetry	<code>telemetry.go</code>	Working
GetSystemStatus	<code>telemetry.go:261</code>	Stub only
BatteryService (all)	<code>battery.go</code>	Complete (80%+ coverage)
ConfigurationService	<code>configuration.go</code>	Missing <code>SetRetransmitConfig</code>
FirmwareUpdateService	<code>firmware.go</code>	ALL 10 stubs
SetPIDGains	<code>gateway_service.go:205</code>	Silently drops

ROS2 Gaps (star-ros2)

The ROS2 stack has solid infrastructure for communication but lacks autonomous operation capabilities (SLAM, navigation). The most urgent gap is validating the SPI bridge on real hardware.

Gap	Severity	Package	Effort
star_spi_bridge hardware testing	CRITICAL	star_spi_bridge	4–8 hrs on HW
Zero-velocity safety shutdown	HIGH	star_spi_bridge	2 hrs
Safety monitor test completion	HIGH	star_safety_monitor	4 hrs
star_bringup launch files	HIGH	star_bringup	8–16 hrs
SetPIDGains custom service type	MEDIUM	star_gateway_bridge	2 hrs

RPLiDAR C1 driver node	HIGH	(new package)	4-8 hrs
EKF sensor fusion	HIGH	star_bringup	4-8 hrs
4-wheel kinematics fix	MEDIUM	star_spi_bridge	2-4 hrs
SLAM (RTAB-Map)	MEDIUM	star_bringup	2-3 days
Navigation stack (Nav2)	LOW	star_bringup	3-5 days

Gap 1: star_spi_bridge Hardware Testing (CRITICAL)

The code is written (420-line lifecycle node + 152-line SPI driver + 397-line tests) but has **never been run on real hardware** (RPi5 + RX72N). This is the critical path for first robot motion.

Hardware test commands:

```
# On RPi5 with ROS2 Jazzy:
cd /workspaces/STAR/star-ros2
colcon build --packages-select star_spi_bridge

source install/setup.bash
ros2 launch star_spi_bridge star_spi_bridge.launch.py

# In second terminal -- send velocity command:
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
  "linear: {x: 0.1}" --rate 10 --times 30

# Verify encoder feedback:
ros2 topic echo /odom/unfiltered --once
ros2 topic hz /joint_states # Expected: ~100 Hz
```

Listing 8: SPI bridge hardware validation procedure

Known risks: SPI device path (/dev/spidev0.0 vs /dev/spidev0.1), SPI mode mismatch, CRC byte order.

Gap 2: Zero-Velocity Safety Shutdown (HIGH)

star_spi_driver_node.cpp:112 has TODO: “Send final zero-velocity frame before deactivation.” If the ROS2 node crashes, motors continue at their last commanded velocity – a critical safety issue.

Fix location: StarSpiDriverNode::on_deactivate() – send zero velocity frame with 3 retry attempts before deactivating publishers.

Gap 3: star_bringup Launch Files (HIGH, 5% Complete)

The star_bringup package is an empty skeleton. Without launch files, each node must be started manually.

Required files:

- launch/star_system.launch.py – main entry point
- launch/spi_bridge.launch.py – lifecycle node management

- launch/safety_monitor.launch.py - safety system
- launch/sensors.launch.py - LiDAR + IMU (when available)
- config/hardware_params.yaml - motor model, wheel geometry
- config/ekf_config.yaml - EKF sensor fusion
- rviz/star_default.rviz - default visualization

ROS2 Package Status Summary

Package	%	Notes
star_gateway_bridge	100	Telemetry push (10 Hz), teleop poll (50 Hz) working
star_spi_bridge	85	Code complete, hardware untested, zero-vel fix needed
star_safety_monitor	40	Core logic works, 3 safety tests skipped
star_bringup	5	Empty skeleton, no launch files
SLAM / Nav2	0	RTAB-Map, robot_localization, Nav2 not configured

UI Gaps (star-ui, TypeScript/React)

The UI only implements gamepad control via WebSocket. This is a proof-of-concept. All operator situational awareness (telemetry, battery, E-Stop) is missing. The fundamental problem is that **browsers cannot access gRPC directly**.

Gap	Severity	Effort
gRPC-Web bridge (WebSocket endpoints in gateway)	CRITICAL	4-8 hrs
Emergency stop button	CRITICAL	3-4 hrs
Telemetry display	CRITICAL	10-15 hrs
Battery status display	CRITICAL	5-8 hrs
Motor status display	HIGH	8-12 hrs
Configuration panel (PID tuning)	HIGH	15-20 hrs
UI tests (vitest)	HIGH	10-15 hrs
State management (Zustand)	MEDIUM	4-6 hrs
Camera feed	MEDIUM	8-12 hrs
LiDAR visualization	LOW	15-20 hrs
Navigation map	LOW	20-30 hrs

Total estimated effort: 110-175 hours for a complete operator UI.

The Core Problem: Browser Cannot Access gRPC

Current: UI WebSocket → /ws/controller (only endpoint)

Recommended fix (Option A - simplest): Add new WebSocket endpoints to the Gateway for streaming telemetry, battery, and encoder data. Gateway converts protobuf to JSON and streams over WebSocket.

```
mux.HandleFunc("/ws/telemetry", s.handleWebSocketTelemetry)
mux.HandleFunc("/ws/battery", s.handleWebSocketBattery)
mux.HandleFunc("/ws/encoders", s.handleWebSocketEncoders)
```

Listing 9: Gateway WebSocket endpoints to add (app/gateway.go)

Gap 1: Emergency Stop Button (CRITICAL - Safety)

No E-Stop button exists. **This is a safety-critical gap.** The implementation requires double-click confirmation to prevent accidental triggering, and a gateway HTTP endpoint POST /emergency-stop.

Gap 2: Telemetry Display (CRITICAL)

The operator cannot see motor encoder counts, velocities, obstacle distances, temperature, or link quality. The component requires the WebSocket telemetry endpoint from the gateway (Gap above).

UI Component Status

Component / Hook	Status
ControllerView.tsx (gamepad)	Working
EmergencyStopButton.tsx	Missing
BatteryStatus.tsx	Missing
TelemetryDisplay.tsx	Missing
MotorStatus.tsx	Missing
ConfigurationPanel.tsx	Missing
hooks/useGamepad.ts	Working
hooks/useControllerConnection	Working
hooks/useTelemetryStream.ts	Missing
store/robotStore.ts	Missing
(Zustand)	
UI tests (vitest)	Zero test files

Protocol Buffer Gaps (star-proto)

The proto schemas are comprehensive and correct. The main gaps are in the **nanopb .options files** and a missing FirmwareChunk message type for

OTA.

Gap	Severity	File	Fix
motor_status max_count:2 (should be 4)	HIGH	motor_control.options	Change to max_count:4
CellData cell_mv max_count missing	MEDIUM	battery_management.options	Add options max_count:16
BmsDeviceInfo.chemistry max_size missing	MEDIUM	battery_management.options	Add options max_size:32
CurrentSensorCalibration arrays	MEDIUM	configuration.options	Add max_count:4
FirmwareChunk missing in wire.proto	HIGH	wire.proto	Add new message type

Gap 1: motor_status max_count Mismatch (HIGH - Data Truncation Bug)

star-proto/nanopb/star/v1/motor_control.options currently reads:

```
# Motor status array (2 motors for differential drive) -- WRONG!
star.v1.SetVelocityResponse.motor_status    max_count:2
```

Listing 10: Current (broken) motor_control.options

The firmware manages **4 motors**. Nanopb silently truncates motors 2 and 3. Fix:

```
# Motor status array (4-wheel drive: FL, FR, BL, BR)
star.v1.SetVelocityResponse.motor_status    max_count:4
```

Listing 11: Correct motor_control.options

After this change, run `make proto-gen` and `make proto-gen-firmware` to regenerate.

Gap 2: FirmwareChunk Missing in wire.proto (HIGH)

For OTA to work, add to WireMessage oneof:

```
message FirmwareChunk {
  uint32 session_id = 1; // From BeginUpdateRequest
  uint64 offset      = 2; // Byte offset in firmware image
  bytes  data        = 3; // Chunk payload (max 512B for embedded)
  uint32 chunk_crc   = 4; // CRC-32 of this chunk
  bool   is_last     = 5; // True for final chunk
}
// Add wire.options: star.v1.FirmwareChunk.data: max_size:512
```

Listing 12: FirmwareChunk message to add to wire.proto

CI/CD Gaps

The CI/CD pipeline has solid coverage for proto and gateway, but completely lacks firmware and UI CI. Regressions in firmware or TypeScript would not be detected until manual testing.

Component	Build CI	Test CI	Coverage	Lint
Proto (buf)	Yes	Yes	N/A	Yes
Gateway (Go)	Yes	Yes	65%+	Yes
ROS2	Yes	Yes	N/A	Non-blocking
Firmware (RX72N)	Missing	Missing	N/A	Missing
UI (TypeScript)	Missing	Missing	N/A	Missing
Docs (LaTeX / Doxygen)	Missing	N/A	N/A	N/A
Top-Level Orchestration	Missing	N/A	N/A	N/A

New Workflow: [.github/workflows/firmware.yml](#)

Pragmatic approach (short-term): Run firmware unit tests using host GCC. Cross-compile with GNURX can run locally (GNURX requires Renesas account, blocking CI automation).

```
name: Firmware
on:
  push:
    paths: ['e2-studio-star-rx72n-firmware/**']
jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: sudo apt-get install -y gcc g++ cmake git
      - run: |
          cd e2-studio-star-rx72n-firmware
          ./tests/run_tests.sh
```

Listing 13: firmware.yml simplified – unit tests only

New Workflow: [.github/workflows/ui.yml](#)

```
name: UI
on:
  push:
    paths: ['star-ui/**']
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: {node-version: '22', cache: 'npm'}
      - run: cd star-ui && npm ci
```

```
- run: cd star-ui && npx tsc --noEmit
- run: cd star-ui && npm test -- --run
- run: cd star-ui && npm run build
```

Listing 14: ui.yml – TypeScript build and test

Open Issue Branches Status

All 8 open worktree branches are effectively complete and ready to merge.

Branch	Issue	Title	Status
fix/issue-298	#298	ThreadX stack checking	Ready to merge
fix/issue-299	#299	Event flags removal	Ready to merge
fix/issue-300	#300	Telemetry failover refactoring	Ready to merge
fix/issue-302	#302	BMS SOC threshold config	Ready to merge
fix/issue-303	#303	1-Wire timing nomenclature	Ready to merge
fix/issue-304	#304	Frame decoder resync	Ready to merge
fix/issue-305	#305	Flash size linker fix	Merged to main
fix/issue-338	#338	UTF-8 documentation cleanup	In progress – uncommitted changes

Recommended merge order (to minimize conflicts):

1. #303 – 1-Wire timing (isolated to rx_ds18b20)
2. #298 – Stack checking (isolated to main.c config)
3. #299 – Event flags (affects task files)
4. #304 – Frame decoder (isolated to rx_frame)
5. #302 – BMS SOC config (affects config + bms_monitor_task)
6. #300 – Telemetry failover (affects telemetry_task)
7. #338 – UTF-8 cleanup (cross-cutting, merge last – commit uncommitted changes first)

Critical Path to First Robot Motion

Estimated Total Time: 2-4 days (with hardware available)

All steps below assume: RPi5 with ROS2 Jazzy installed, RX72N con-

nected via SPI, motors wired to DRV8243S, 12V power supply, debug UART accessible via USB-C.

Phase 1 Fix nanopb motor_status Bug (30 min)

Edit `star-proto/nanopb/star/v1/motor_control.options`: change `max_count:2` to `max_count:4`.
Run `make proto-gen` && `make proto-gen-firmware`. Verify firmware builds.

Phase 2 Implement WireMessage Dispatcher (4-6 hrs)

Add `internal_dispatch_wire_message()` to `comm_task.c`.
Add handler functions to `motor_control_task.h` and `motor_control_task.c`.
Write Unity tests in `tests/test_comm_task_dispatch.c`.

Phase 3 Flash Firmware to RX72N (1-2 hrs)

Build with `./build.sh debug`. Flash via e2 studio HardwareDebug configuration.
Verify via UART (`screen /dev/ttyACM0 115200`): look for “ThreadX running”.

Phase 4 Launch Gateway (30 min)

`cd star-gateway && go build ./cmd/star-gateway`
`TRANSPORT_MODE=force-spi ./star-gateway`
Verify: “Transport connected” in logs, PING/PONG frames exchanged.

Phase 5 Test ROS2 SPI Bridge on Hardware (4-8 hrs)

Build and launch `star_spi_bridge` on RPi5.
Send `ros2 topic pub /cmd_vel` and verify motors spin.
Check `ros2 topic echo /odom/unfiltered`.

Phase 6 Zero-Velocity Safety Fix (2 hrs)

Implement `on_deactivate()` zero-velocity frame in `star_spi_driver_node.cpp`:

Phase 7 Add Telemetry WebSocket to Gateway (4-6 hrs)

Add `/ws/telemetry`, `/ws/battery`, `/ws/encoders` endpoints to gateway.
Connect UI to stream robot state.

Phase 8 Add Emergency Stop to UI (3 hrs)

Implement `EmergencyStopButton` with double-click confirmation.
Add `POST /emergency-stop` gateway endpoint.

Phase 9 Launch Gateway Bridge (1 hr)

`ros2 run star_gateway_bridge star_gateway_bridge_node`
Verify full stack: UI → Gateway → ROS2 → SPI → RX72N → Motors

Success Criteria for First Motion:

- `ros2 topic pub /cmd_vel` causes physical motors to spin
- Encoder feedback visible on `/odom/unfiltered` and `/joint_states`
- Battery state visible on `/battery_state`
- UI gamepad control causes robot to move
- UI shows telemetry (motor velocities, encoder counts)
- E-Stop button stops motors within <500ms

- Robot stops when ROS2 node deactivates (zero-velocity frame sent)

Hardware Gaps

Hardware	Status	Driver	Notes
4x DRV8243S Motor Drivers	Complete	rx_drv8243	-
4x Quadrature Encoders	Complete	rx_encoder	MTU/TPU
4x HC-SR04 Sonar (IRQ)	Complete	rx_hcsr04	IRQ8-IRQ11
DS18B20 Temperature	Complete	rx_ds18b20	1-Wire
BQ4050 BMS Fuel Gauge	Partial	rx_bq4050	Faults only, no telemetry
6x Status LEDs	Complete	GPIO	-
Host SPI (RSPi2 @ 10 Mbps)	Complete	rx_spi_comm	-
USB CDC (3 virtual COM)	Complete	rx_usb + rx_usb_comm	-
Debug UART (SCI9)	Complete	rx_core	115200 baud
E-Stop (GTETRQ/POEG)	Complete	rx_poeg	-
BMS ALERT IRQ (IRQ13)	Complete	rx_bms_alert	PR #333
RPLiDAR C1	Missing	None	UART interface needed
Host I2C (RIIC0)	Reserved	rx_hal (low-level)	P12/P13 allocated
IWDT Watchdog	Complete	rx_iwtd	100 Hz monitor

PCB Manufacturing Status: Gerbers are generated for all 3 boards (MCU, Power, BMS). Manufacturing status unknown - verify if boards have been fabricated and assembled.

Hardware Pinout Verification Checklist

Before first hardware bring-up, verify the following connections against docs/sections/03_hardware_pinout.tex:

- ☐ Motor FL: GTIOCA0/GTIOCB0 → H-bridge inputs confirmed?
- ☐ Motor FR: GTIOCA1/GTIOCB1 → confirmed?
- ☐ Encoder signals: MTU channels match firmware config?
- ☐ RSPi2 CS/SCK/COPI/CIPO connected to RPi5 SPI pins?
- ☐ RIIC1 connected to BQ4050 BMS IC?
- ☐ BMS ALERT connected to P05/IRQ13?
- ☐ 4x HC-SR04 ECHO pins match IRQ8-IRQ11?
- ☐ SCI9 TX/RX accessible via USB-C debug cable?

Testing Gaps

Test Category	Status	Coverage	Gap
Firmware Unit Tests (Unity)	87 files	High	Not run in CI
Gateway Unit Tests (Go)	30+ files	65%+	Complete
Proto Serialization Tests	Complete	All targets	-
ROS2 Unit Tests	Partial	1318 LOC	3 tests skipped
UI Unit Tests	None	0%	Zero test files
Virtual RX72N Tests	432 LOC	Frame protocol	No motor dynamics
Firmware Integration Tests	None	0%	Full-path test missing
ROS2 Integration Tests	None	0%	No E2E tests
Hardware-in-Loop Tests	None	0%	Requires physical HW

Testing Priority Matrix

Test	Safety Critical	CI Automated	Effort
UI EmergencyStop tests	Yes	No	3 hrs
Firmware WireMessage dispatch tests	Yes	No	3 hrs
ROS2 safety monitor tests (3 skipped)	Yes	No	4 hrs
E2E E-Stop latency test (<100ms)	Yes	No	4 hrs
Motor dynamics simulation	No	No	1-2 days
Gateway HIL test enhancements	No	Partial	4-8 hrs
Transport failover E2E	No	No	4-8 hrs
UI component tests	No	No	10-15 hrs
Hardware validation suite	No	No (HW req'd)	2-3 days

Local Test Runner (All Components)

```
#!/bin/bash
set -e

echo "=== Firmware Unit Tests ==="
cd e2-studio-star-rx72n-firmware && ./tests/run_tests.sh

echo "=== Gateway Tests ==="
```

```

cd ../star-gateway && go test ./... -count=1

echo "=== Proto Tests ==="
cd ../star-proto/tests/go && go test ./...

echo "=== ROS2 Tests ==="
cd /workspaces/STAR/star-ros2
colcon test --return-code-on-test-failure

echo "=== UI Tests ==="
cd /workspaces/STAR/star-ui && npm test -- --run

echo "All tests passed!"

```

Listing 15: scripts/run_all_tests.sh – run all subsystem tests

Virtual RX72N Simulator Enhancements

The virtual RX72N simulator (star-gateway/cmd/virtual_rx72n/) is a well-implemented Go-based TCP/Unix socket server (525 lines + 432 lines of tests). It handles the full frame protocol and provides basic telemetry simulation. Motor dynamics simulation is the most critical missing feature.

Enhancement	Priority	Effort	Unlocks
Motor dynamics simulation	HIGH	1-2 days	Odometry testing, PID validation HARQ/FEC testing, resilience Battery depletion testing PID tuning end-to-end Concurrent UI connections Temperature sensor testing
Fault injection API	HIGH	4-8 hrs	
BMS simulation	MEDIUM	4-6 hrs	
Config message handling	MEDIUM	2-3 hrs	
Multi-client support	LOW	4-6 hrs	
Thermal simulation	LOW	4-6 hrs	

Motor Dynamics Model

From MATLAB system identification (matlab/motor_model_1st_order.m):

$$G(s) = \frac{K}{\tau s + 1} = \frac{3.665}{0.075s + 1}$$

Discrete-time (backward Euler, $T = 0.01$ s at 100 Hz):

$$v[k] = (1 - e^{-T/\tau}) \cdot K \cdot u[k] + e^{-T/\tau} \cdot v[k - 1]$$

$$\approx 0.1248 \cdot 3.665 \cdot u[k] + 0.8752 \cdot v[k - 1]$$

```
// star-gateway/cmd/virtual_rx72n/motor_sim.go
type MotorSim struct {
    velocityRadps float64    // Current angular velocity (rad/s)
    positionRad    float64    // Accumulated angle (rad)
    encoderTicks   atomic.Int64
    mu             sync.Mutex
}

func (m *MotorSim) Update(pwmDuty float64) {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Clamp duty cycle to [-1.0, 1.0]
    pwmDuty = math.Max(-1.0, math.Min(1.0, pwmDuty))

    // First-order discrete-time system (backward Euler)
    const K    = 3.665    // DC gain: rad/s per volt
    const tau  = 0.075    // Time constant: seconds
    const T    = 0.010    // Sample period: 100 Hz
    const Vs   = 12.0     // Supply voltage

    alpha := math.Exp(-T / tau)
    inputVoltage := pwmDuty * Vs
    m.velocityRadps = (1-alpha)*K*inputVoltage + alpha*m.velocityRadps

    m.positionRad += m.velocityRadps * T
    const ticksPerRev = 1364 // 341 PPR * 4 (quadrature)
    m.encoderTicks.Store(int64(m.positionRad / (2 * math.Pi) * ticksPerRev))
}
```

Listing 16: Motor simulation Go implementation

Fault Injection API

```
type FaultConfig struct {
    CrcErrorRate      float32    `json:"crc_error_rate"`
    PacketLossRate     float32    `json:"packet_loss_rate"`
    LatencySpikeDuration time.Duration `json:"latency_spike_ms"`
    LatencySpikeRate   float32    `json:"latency_spike_rate"`
}

// HTTP endpoint: POST /fault    (set faults)
//                DELETE /fault  (clear faults)
//                GET /status     (simulator state)
```

Listing 17: Fault injection HTTP control endpoint

Implementation Roadmap

Immediate Actions (This Week)

1. **Merge open issues #298, #299, #300, #302, #303, #304** (all complete – create PRs)
2. **Finalize #338:** Commit uncommitted changes, create PR, merge
3. **Fix nanopb bug:** `motor_status max_count:2 → max_count:4` in `motor_control`.
4. **Implement WireMessage dispatcher** in `comm_task.c` (4-6 hrs)
5. **Add firmware CI** (`.github/workflows/firmware.yml`) – unit tests in CI

Short-Term (Next 2-4 Weeks)

6. Flash firmware to RX72N and verify boot sequence via UART
7. Test ROS2 SPI bridge on real hardware – first motor motion
8. Add WebSocket telemetry endpoints to gateway
9. Add EmergencyStopButton to UI + gateway endpoint
10. Implement BMS telemetry (SOC/voltage in telemetry stream)
11. Fix zero-velocity safety shutdown in `on_deactivate()`
12. Add UI CI workflow (`.github/workflows/ui.yml`)

Medium-Term (1-2 Months)

13. Write UI tests (vitest) for all components
14. Implement NVS configuration persistence (Data Flash)
15. Implement `star_bringup` launch files
16. Add motor dynamics to `virtual_rx72n` simulator
17. Implement SetPIDGains forwarding (gateway → RX72N)
18. Add EKF sensor fusion (`robot_localization` package)
19. Configure RPLiDAR C1 ROS2 node (`rplidar_ros` package)

Long-Term (2-6 Months)

20. Implement FirmwareUpdateService (OTA) – gateway + firmware
21. Configure SLAM (RTAB-Map + `robot_localization`)
22. Set up Nav2 for autonomous navigation
23. Set up hardware-in-loop CI with self-hosted RPi5 runner
24. UI advanced: camera feed, LiDAR visualization, navigation map

Appendix: System Architecture

Communication Flow

Source	Transport	Destination
UI (TypeScript)	WebSocket	Gateway (Go on RPi5)
Gateway (Go)	gRPC (port 50051)	ROS2 star_gateway_bridge
ROS2	/cmd_vel topic	star_spi_bridge node
star_spi_bridge	SPI @ 10 Mbps + nanopb	RX72N MCU
RX72N	SPI + nanopb TelemetryData	Gateway (telemetry back)

Firmware Task Priority Table

Priority	Task	Rate	Stack
5	Comm Task (SPI/USB receive)	100 Hz	4096B
6	Telemetry Task (SPI/USB send)	20 Hz	2048B
7	BMS Monitor Task	1 Hz	2048B
8	Motor Control Task (PID loop)	100 Hz	4096B
9	Obstacle Detection Task	50 Hz	2048B
10	Temp Sensor Task (DS18B20)	1 Hz	2048B
11	Watchdog Monitor Task	100 Hz	1024B
12	LED Status Task	20 Hz	1024B
13	App Main Task (idle)	N/A	2048B

Motor PID Parameters (from MATLAB)

Motor transfer function: $G(s) = \frac{3.665}{0.075s+1}$ (DC gain = 3.665 rad/s/V, $\tau = 75$ ms)

PID controller designed via pole placement:

- $K_p = 0.286$
- $K_i = 8.01$ rad/s per Vs
- $K_d = 0$ (first-order plant, derivative not needed)
- Closed-loop bandwidth: ≈ 13 Hz

SPI Frame Protocol

Field	Size	Value
SYNC	2 bytes	0x55AA
SEQ	2 bytes	Sequence number (wraps at 0xFFFF)
LEN	2 bytes	Payload length
TYPE	1 byte	Frame type (COMMAND, ACK, TELEMETRY, ...)
FLAGS	1 byte	FlagRequiresAck, FlagFEC, ...
PAYLOAD	var	nanopb-encoded protobuf
CRC-32	4 bytes	IEEE 802.3, LSB-first